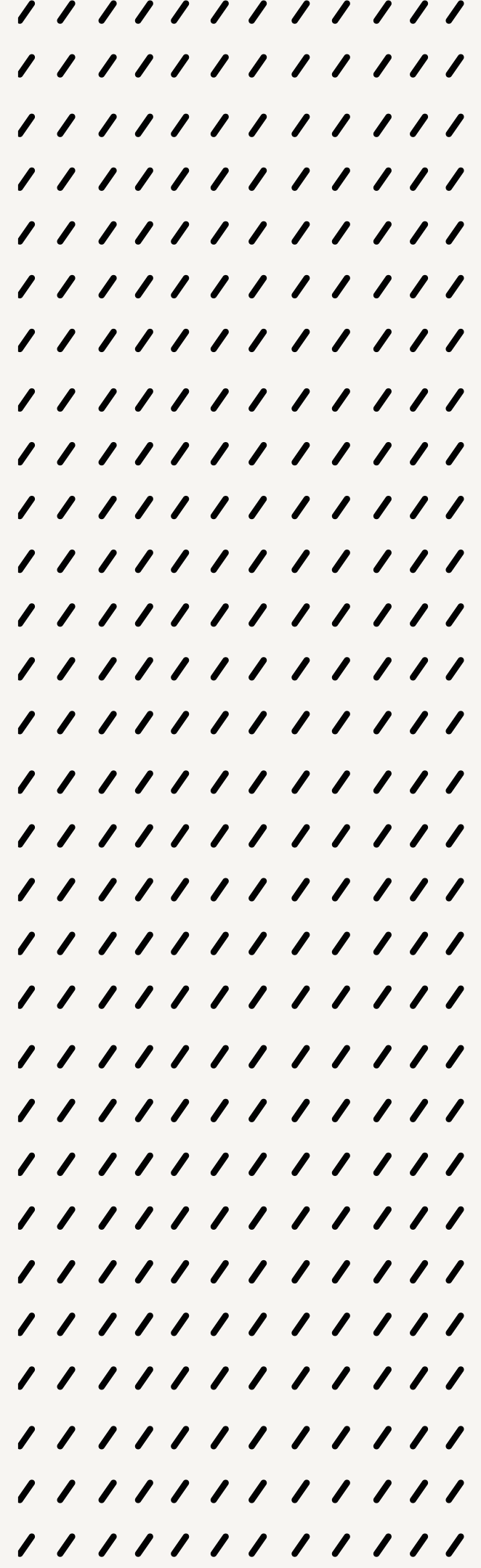


Manual de Usuario

PROYECTO 2



Contenido

- 3 **Propósito**
- 4 **Alcance**
- 5 **Descripción del Sistema**
- 6 **Características**
Principales
- 7 **Requerimientos o**
Funcionalidades
- 8 **Usabilidad y**
Navegabilidad

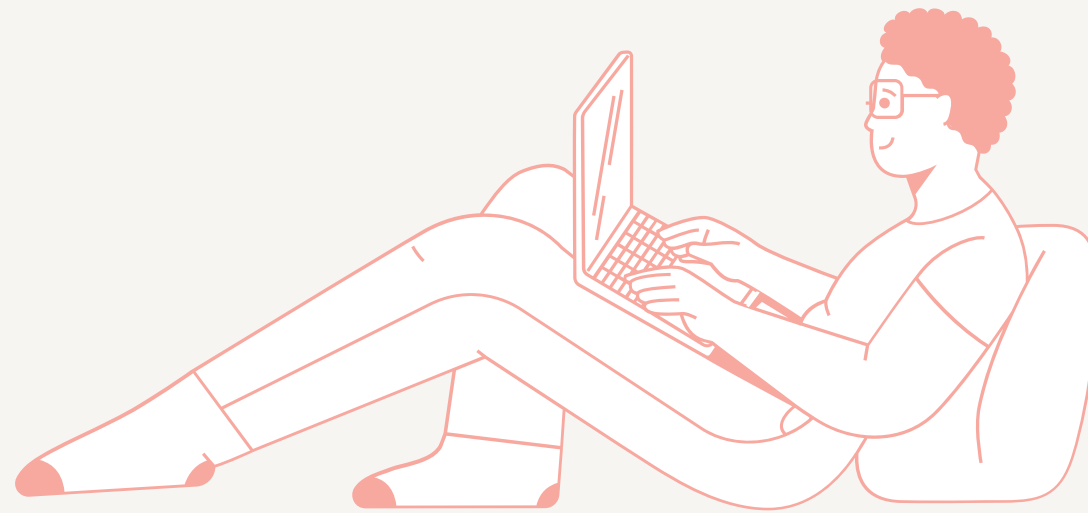
Propósito



El propósito de este manual es ayudar al usuario a utilizar y entender el software **GoScript** que analiza el lenguaje ingresado por el usuario y genera salidas.

En este manual se presentará la información necesaria para utilizar correctamente el software y solucionar posibles problemas que se lleguen a presentar.

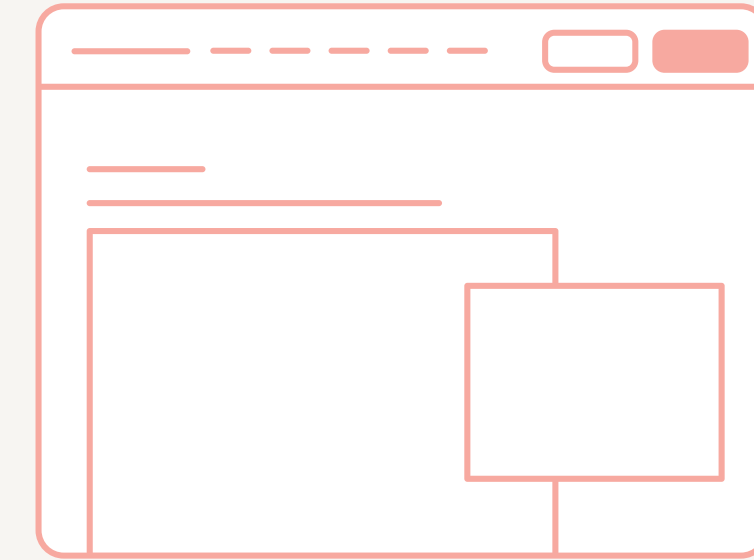
Alcance



Este manual de usuario esta dirigido a los usuarios del software GoScript.

Los conocimientos necesarios para utilizar este software son conocimientos básicos de computación y conocimientos básicos de manejo de archivos.

Descripción del Sistema



El software esta diseñado para poder cargar un documento tipo GoScript y mostrarlo en el editor de archivos, también cuenta con la función de crear un archivo desde cero, editar y guardar diferentes archivos tipo GoScript.

Luego de comprobar que el documento cumple con la estructura correcta, el usuario puede ejecutar el código para que el programa analice el lenguaje y genere los reportes correspondientes (Tabla de Tokens, Tabla de Errores y Tabla de Símbolos).

Durante la ejecución también se generaran las salidas correspondientes según las instrucciones y se mostrarán en la consola del programa junto con los errores encontrados durante la ejecución.

Características Principales

01

**Análisis de
Lenguaje
GoScript**

02

**Ejecución de
instrucciones
y generación
de salidas.**

03

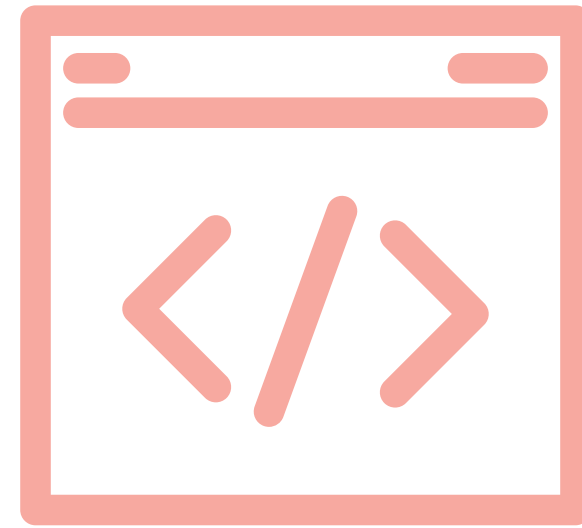
**Generación
de Reportes**

04

**Edición de
Archivos .gst**

Requerimientos

Para utilizar este software solo es necesario ejecutar GoScript en una computadora con sistema operativo windows, un navegador web y un archivo .gst con las instrucciones correctas en el lenguaje GoScript.

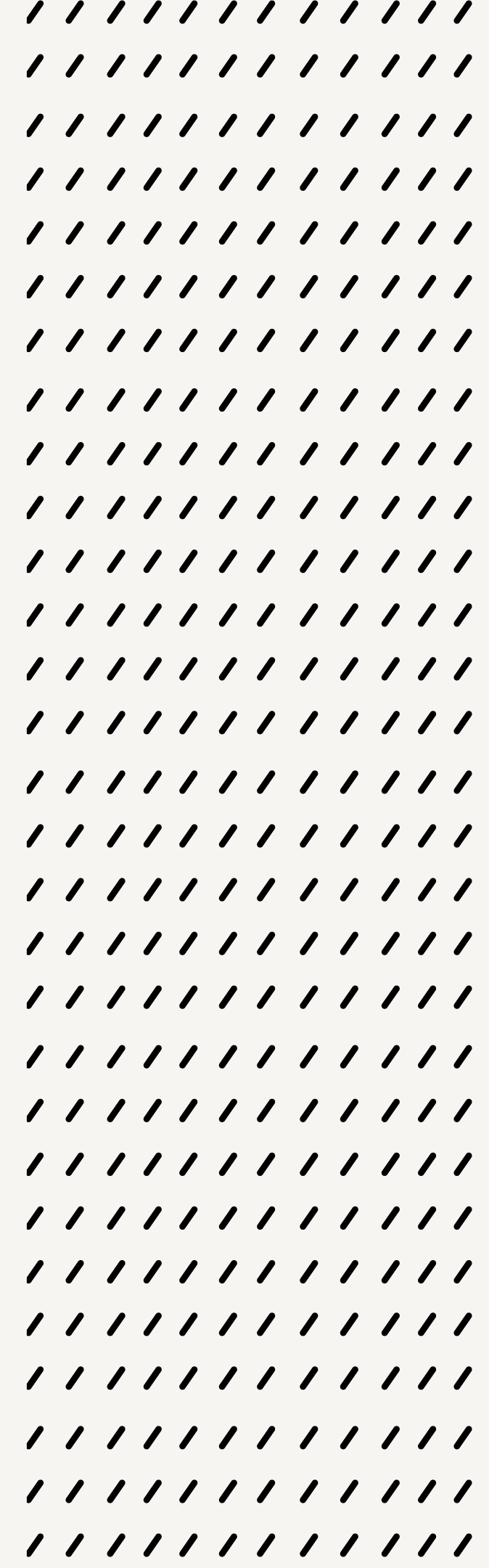


Archivo .gst



Software GoScript

Usabilidad y Navegabilidad

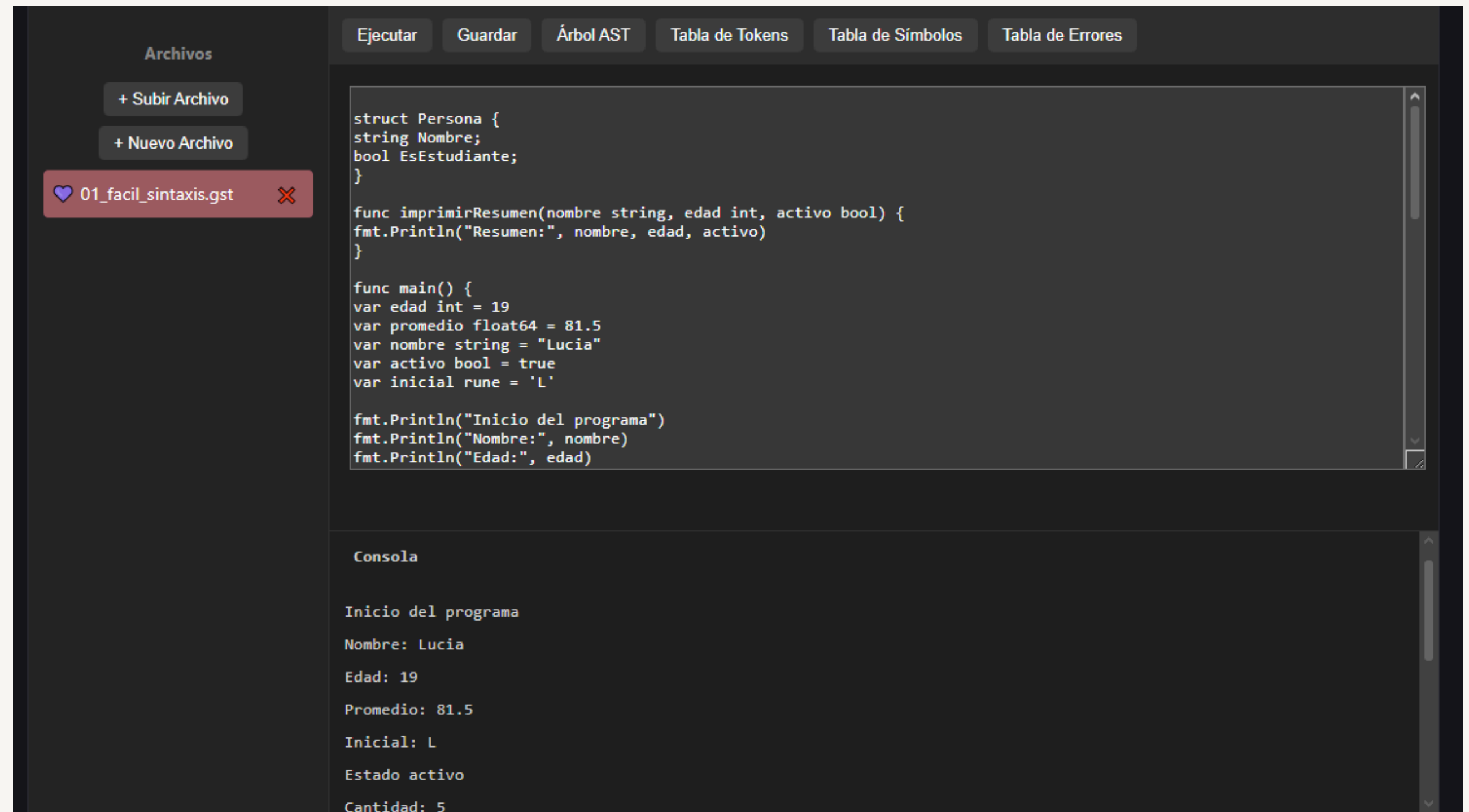


Pantalla Principal

Archivo: Permite administrar los archivos con las opciones "Nuevo Archivo", "Subir Archivo", "Guardar" y "Eliminar".

Reportes: Muestra los reportes generados luego de ejecutar el lenguaje.

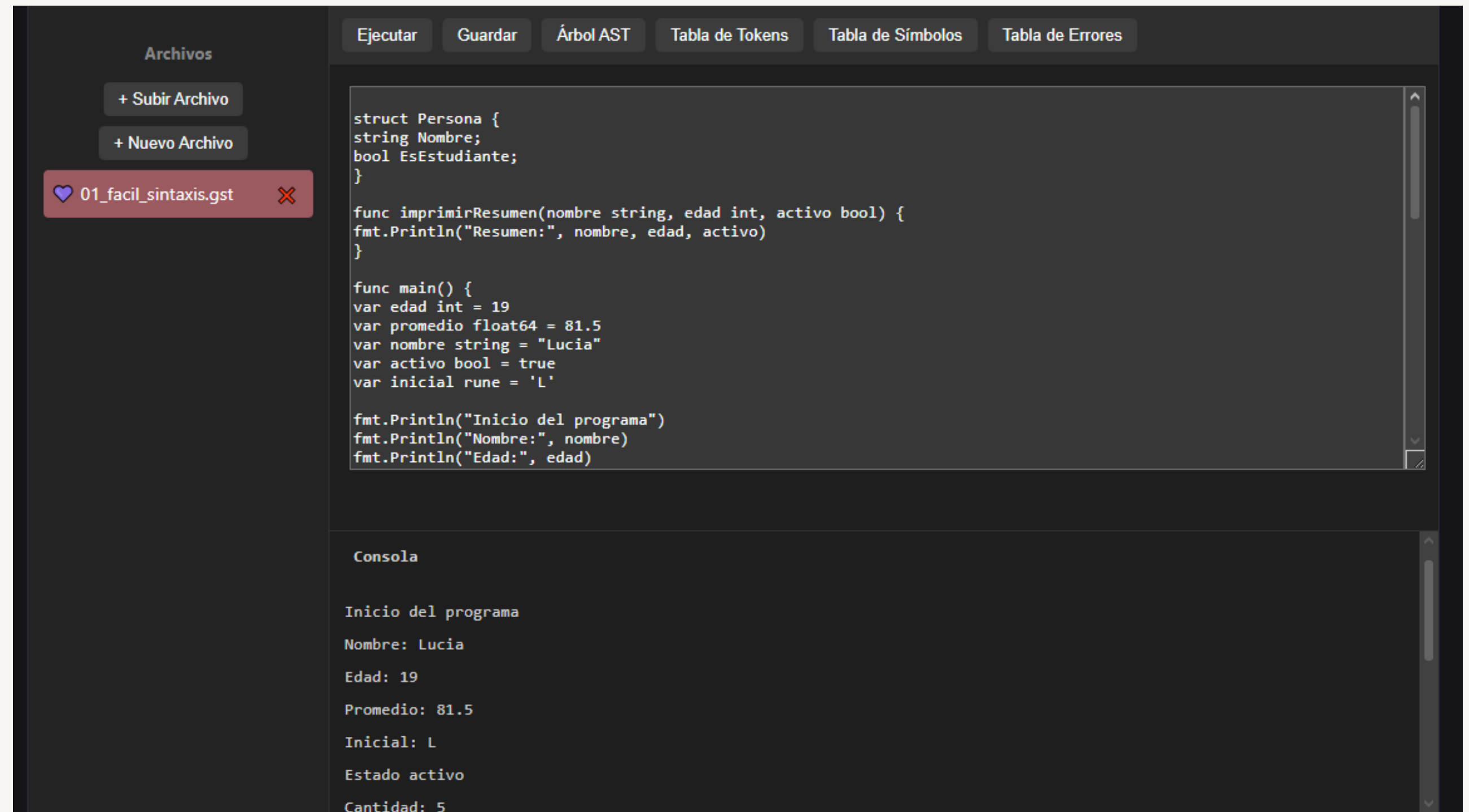
Ejecutar: Analiza el lenguaje.



Pantalla Principal

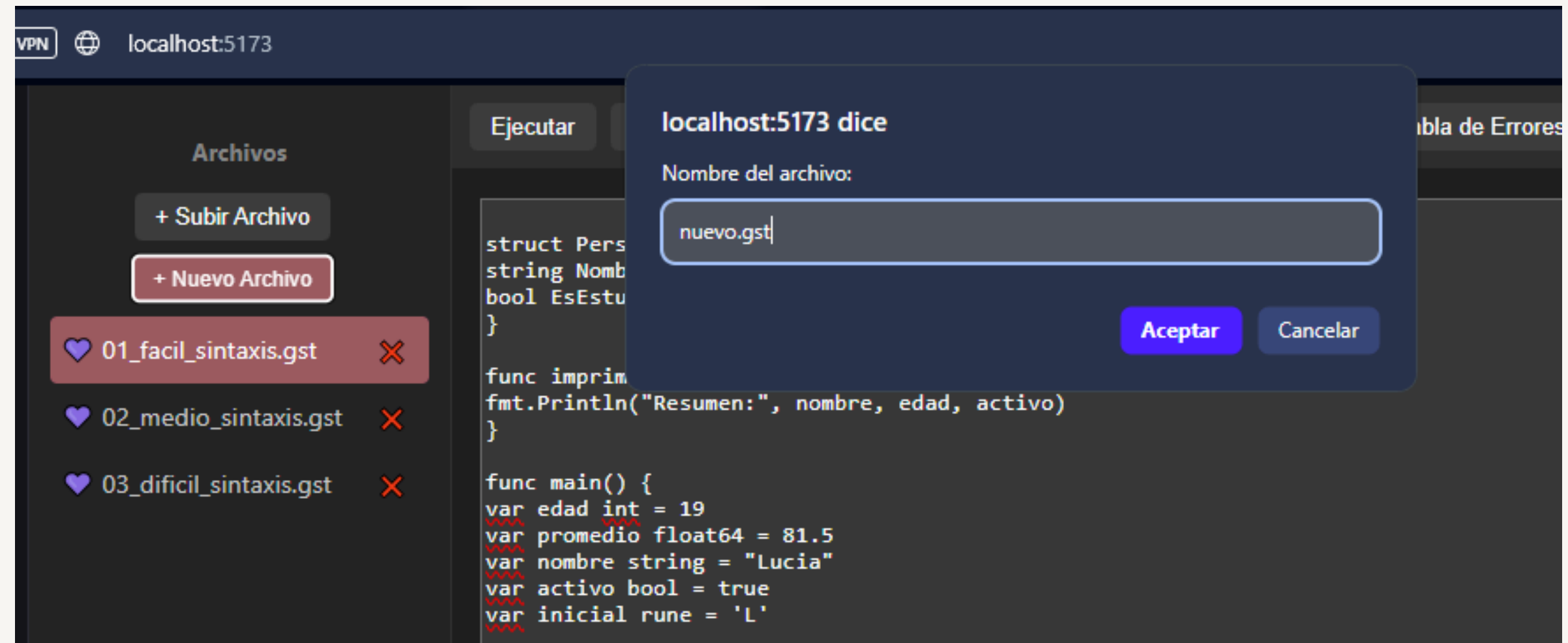
EDITOR: Muestra el contenido del archivo .gst elegido.

CONSOLA: Muestra el estado de la ejecución del lenguaje, las salidas y los errores generados.



Nuevo Archivo

Al presionar el botón de "Nuevo Archivo" se muestra una ventana en la que se debe indicar el nombre del nuevo archivo que se desea crear y para guardarlo solo se debe seleccionar el botón de "Guardar".



Reportes

Luego de ejecutar el lenguaje se puede presionar el botón de "Tabla de Errores", "Tabla de Tokens" ó "Tabla de Símbolos" según se desee visualizar.

< > ↺ VPN 📄 file:///C:/Users/Karen/OneDrive/Documentos/PRIME

Reporte de Errores

No.	Tipo	Descripción	Fila	Columna
1	Léxico	El carácter ? no pertenece al lenguaje	5	1
2	Léxico	El carácter ? no pertenece al lenguaje	5	2

Reporte de Tokens

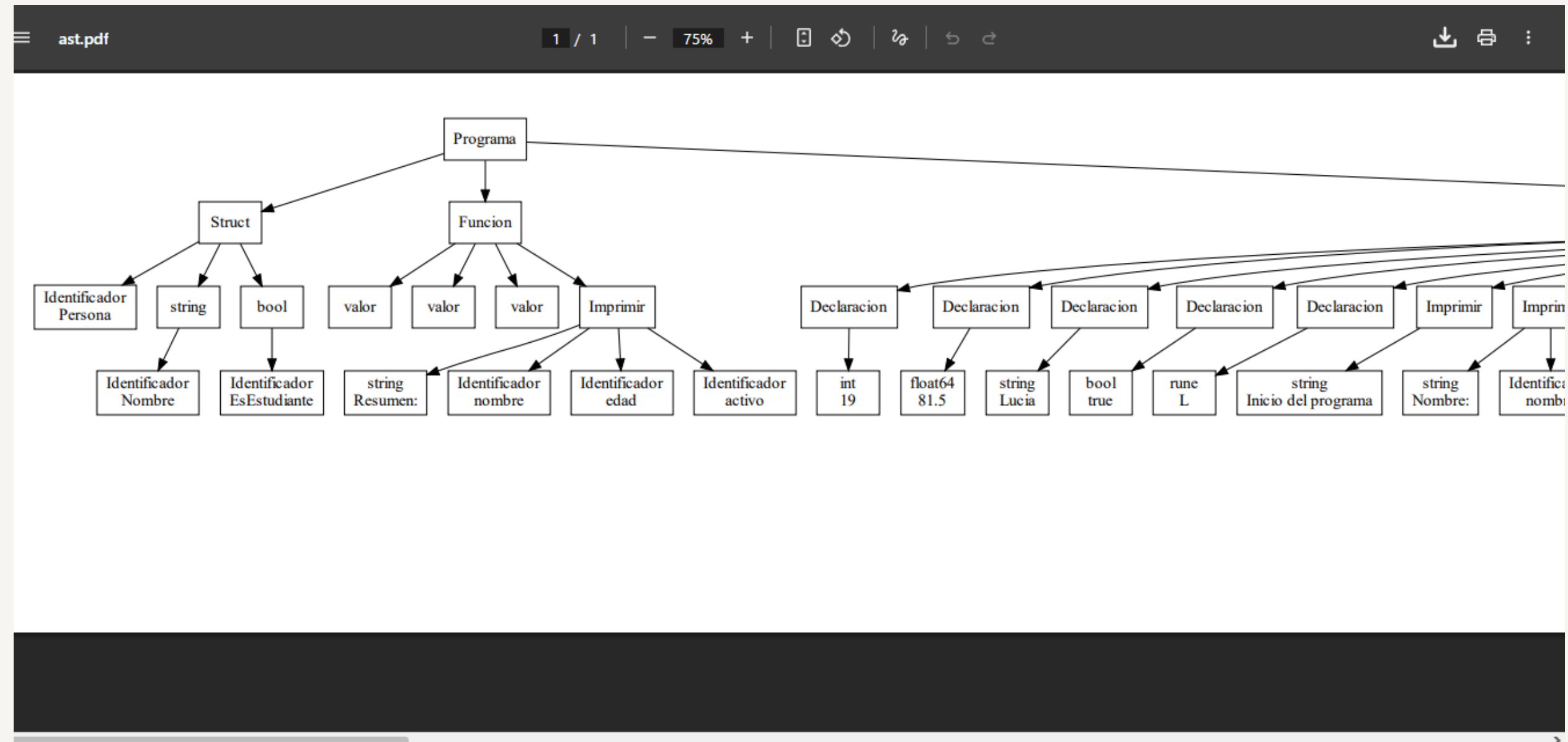
No.	Tipo	Lexema	Fila	Columna
1	struct	STRUCT	1	0
2	Persona	ID	1	7
3	{	LLAVE_A	1	15
4	string	STRING	2	0
5	Nombre	ID	2	7

Reporte de Símbolos

No.	ID	Tipo Simbolo	Tipo Dato	Ambito	Fila	Columna
1	Persona	Struct	struct		2	7
2	imprimirResumen	Función	Función	Global	7	5
3	edad	Variable	int	imprimirResumen	12	4
4	promedio	Variable	float64	imprimirResumen	13	4
5	nombre	Variable	string	imprimirResumen	14	4
6	activo	Variable	bool	imprimirResumen	15	4
7	inicial	Variable	rune	imprimirResumen	16	4
8	numeros	Slice	[]int	imprimirResumen	34	0
9	main	Función	Función	Global	11	5

Árbol AST

Al seleccionar el botón "Árbol AST" se muestra el AST del lenguaje analizado.



Archivo .gst

Para analizar el lenguaje se puede subir o crear archivos .gst dentro del programa con las instrucciones que se desea ejecutar.

```
01_facil_sintaxis.gst X
C: > Users > Karen > Downloads > 01_facil_sintaxis.gst
1
2 struct Persona {
3     string Nombre;
4     bool EsEstudiante;
5 }
6
7 func imprimirResumen(nombre string, edad int, activo bool) {
8     fmt.Println("Resumen:", nombre, edad, activo)
9 }
10
11 func main() {
12     var edad int = 19
13     var promedio float64 = 81.5
14     var nombre string = "Lucia"
15     var activo bool = true
16     var inicial rune = 'L'
17
18     fmt.Println("Inicio del programa")
19     fmt.Println("Nombre:", nombre)
20     fmt.Println("Edad:", edad)
21     fmt.Println("Promedio:", promedio)
22     fmt.Println("Inicial:", inicial)
23
24     edad = edad + 1
25     promedio += 3
26     nombre = nombre + " Perez"
27
28     if activo {
29         fmt.Println("Estado activo")
30     } else {
31         fmt.Println("Estado inactivo")
32     }
```